



Chapter_10

2.1 자연어 처리란

- 우리가 평소에 쓰는 말 : 자연어
 - 자연어 처리 (NLP) → 우리가 평소에 쓰는 말을 어떻게 처리하는지?
⇒ 우리의 말을 컴퓨터에게 이해시키기 위한 기술*분야
- 언어가 가진 '부드러움'을 어떻게 컴퓨터에게 전달할 것인가?

2.1.1 단어의 의미

말 → '문자'

말의 의미 → '단어' 로 구성된다! *따라서 단어는 의미의 최소 단위

- 컴퓨터에게 자연어를 이해시킬 때 ⇒ '단어의 의미'를 이해시키는 것이 중요하다

컴퓨터에게 '단어의 의미' 이해시키기 *잘 파악하는 표현 방법

1. 시소러스를 활용한 기법
2. 통계기법 기법
3. 추론 기반 기법

2.2 시소러스

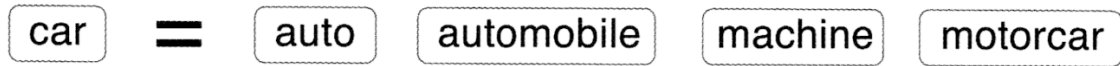
'단어의 의미' → 사람이 직접 단어의 의미를 정의하는 방식

→ 지금까지 단어의 의미를 정의하기 위한 수많은 시도가 이루어졌지만, 사람이 이용하는 일반적인 사전이 아니라 **시소러스 형태의 사전**을 사용

시소러스?

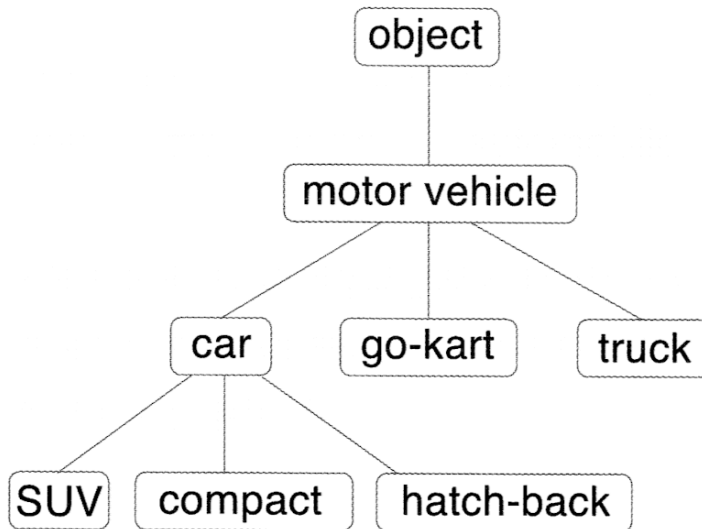
→ 유의어 사전, '뜻이 같은 단어' (동의어), '뜻이 비슷한 단어' (유의어) 가 한 그룹으로 분류되어 있다.

그림 2-1 동의어의 예: “car”, “auto”, “automobile” 등은 “자동차”를 뜻하는 동의어다.



- 시소러스에서는 단어 사이의 ‘상위와 하위’ 혹은 ‘전체와 부분’ 등 더 세세한 관계까지 정의해둔 경우가 있다.

그림 2-2 단어들을 의미의 상·하위 관계에 기초해 그래프로 표현한다(문헌 [14]를 참고하여 그림).



→ 이처럼 모든 단어에 대한 유의어 집합을 만든 다음, 단어들의 관계를 그래프로 표현하여 단어 사이의 연결을 정의

- 이와 같은 ‘단어 네트워크’를 이용하여 컴퓨터에게 단어 사이의 관계를 가르친다.

→ 이와 같은 방식으로 단어의 의미를 ‘간접적으로’ 부여

2.2.1 WordNet

가장 유명한 시소러스 → WordNet

WordNet

- 유의어를 얻거나 ‘단어 네트워크’를 이용할 수 있다.
 - 단어 네트워크를 통한 단어 사이의 유사도를 구할 수 있다!

2.2.2 시소러스의 문제점

시소러스 → 수많은 단어에 대한 동의어와 계층 구조 등의 관계가 정의되어있다!

- 이 지식을 활용하여 ‘단어의 의미’를 간접적으로 컴퓨터에게 전달

But ! *사람이 수작업으로 레이블링하는 방식에는 큰 결점이 존재

1. 시대 변화에 대응하기 어렵다.
2. 사람을 쓰는 비용이 크다.
3. 단어의 미묘한 차이를 표현할 수 없다.

⇒ 시소러스 (단어의 의미를 사람이 정의하는 기법) 에는 문제가 존재한다.

2.3 통계 기반 기법

- 위의 시소러스의 문제점을 피하기 위한 방법!

*말뭉치 를 사용

- 말뭉치란?
 - 대량의 텍스트 데이터 (자연어 처리 연구나 애플리케이션을 염두에 두고 수집된 텍스트 데이터)
 - 사람이 쓴 글
 - 말뭉치에는 자연어에 대한 사람의 '지식'이 담겨 있다.
 - 문장을 쓰는 방법, 단어를 선택하는 방법, 단어의 의미 등 자연어에 대한 사람의 지식이 포함

⇒ 이렇게 사람의 지식으로 가득한 말뭉치에서 자동으로, 효율적으로 핵심을 추출

2.3.1 파이썬으로 말뭉치 전처리하기

```
text='You say goodbye and I say hello.'
text=text.lower()
text=text.replace('.', ' .')
text
'you say goodbye and i say hello .'
```

- 이 텍스트를 단어 단위로 분할

```
words=text.split(' ')
words
['you', 'say', 'goodbye', 'and', 'i', 'say', 'hello', '.']
```

- 모든 문자를 소문자로 변환 후 단어 단위로 분할

- 하지만 단어를 텍스트 그대로 조작하는 것은 불편
 - 단어에 ID를 부여하고, ID의 리스트로 이용할 수 있도록 한 번 더 처리

```
word_to_id={}
id_to_word={}
for word in words:
    if word not in word_to_id:
        new_id=len(word_to_id)
        word_to_id[word]=new_id
        id_to_word[new_id]=word
```

- 파이썬의 딕셔너리를 이용하여 단어 ID와 단어를 짝지어주는 대응표 작성
 - 반복문들 돌며 단어들 중에서 만약 딕셔너리에 없다면 각각의 딕셔너리에 추가
- id_to_word : 단어 ID에서 단어로의 변환
- word_to_id : 단어에서 단어 ID로의 변환

```
id_to_word
{0: 'you', 1: 'say', 2: 'goodbye', 3: 'and', 4: 'i', 5: 'hello', 6: '.'}

word_to_id
{'you': 0, 'say': 1, 'goodbye': 2, 'and': 3, 'i': 4, 'hello': 5, '.': 6}
```

- 단어 ID와 단어의 대응표 생성

```
id_to_word[1]
'say'

word_to_id['hello']
5
```

- 위의 코드와 같이 대응표를 활용하여 단어와 그에 대응하는 단어 ID를 찾을 수 있다!

```
import numpy as np
corpus=[word_to_id[w] for w in words]
corpus=np.array(corpus)
corpus

array([0, 1, 2, 3, 4, 1, 5, 6])
```

- 파이썬 내포 표기를 사용하여 단어 목록에서 단어 ID 목록으로 변환 → 다시 넘파이 배열로 변환

*내포 표기? ⇒ 리스트나 딕셔너리 등의 반복문 처리를 간단하게 쓰기 위한 기법

```
def preprocess(text):
    text=text.lower()
    text=text.replace('.', ' .')
    words=text.split(' ')
    word_to_id={}
    id_to_word={}
    for word in words:
        if word not in word_to_id:
            new_id=len(word_to_id)
            word_to_id[word]=new_id
            id_to_word[new_id]=word
    corpus=np.array([word_to_id[w] for w in word])
    return corpus,word_to_id,id_to_word
```

- 위의 함수를 사용하여 말뭉치 처리를 다음과 같이 수행

```
text='You say goodbye and I say hello.'
corpus,word_to_id,id_to_word=preprocess(text)
```

- corpus : 단어 ID 목록
- word_to_id : 단어 → 단어 ID
- id_to_word : 단어 ID → 단어

⇒ 말뭉치를 사용해 단어의 의미를 추출

2.3.2 단어의 분산 표현

‘색’을 벡터(3차원 공간의 벡터에서 특정색을 공간 내의 점으로 표현하듯)로 표현하듯, ‘단어’를 벡터로 표현?

‘단어의 의미’를 정확하게 파악할 수 있는 벡터 표현

⇒ 단어의 분산 표현

2.3.3 분포 가설

📌 중요한 아이디어 ⇒ ‘단어의 의미는 주변 단어에 의해 형성된다’

⇒ 분포 가설

- 단어 자체에는 의미 x , 단어가 사용된 ‘맥락’이 의미를 형성

*‘I drink beer’ ‘We drink wine’, 에서 ‘drink’ 주변에는 음료가 등장하기 쉽다.

*‘I guzzle wine’ ‘We guzzle wine’ 이라는 문장에서 ‘guzzle’이라는 단어는 ‘drink’와 같은 맥락

⇒ 2개의 의미는 가까운 의미의 단어이다! (guzzle = 폭음하다)

‘맥락’ → (주목하는 단어) 주변에 놓인 단어

그림 2-3 윈도우 크기가 2인 ‘맥락’의 예. 단어 “goodbye”에 주목한다면, 그 좌우의 두 단어(총 네 단어)를 맥락으로 이용한다.

you say goodbye and i say hello.

- 맥락의 크기 (주변 단어를 몇 개나 포함할지): 윈도우크기_Window Size
(1이면 좌우 1개씩, 2면 좌우 2개씩)

2.3.4 동시발생 행렬

- 분포 가설에 기초하여 단어를 벡터로 나타내는 방법 → 주변 단어를 ‘세어보는’ 방법!
⇒ 어떤 단어에 주목했을 때, 그 주변에 어떤 단어가 몇 번이나 등장하는지를 세어 집계하는 방법
→ ‘통계 기반 기법’

```

text='You say goodbye and I say hello.'
corpus,word_to_id,id_to_word=preprocess(text)

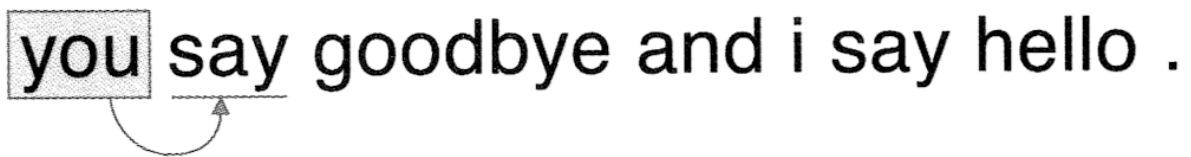
print(corpus)
print(id_to_word)

[6]
{0: 'you', 1: 'say', 2: 'goodbye', 3: 'and', 4: 'i', 5: 'hello', 6: '.'}

```

- 단어 수가 총 7개임을 알 수 있다!
- 단어 "You"를 기준으로 단어의 맥락에 해당하는 단어의 빈도 계산 (윈도우 크기 : 1)

그림 2-4 단어 "you"의 맥락을 세어본다.



- "You"의 맥락은 "Say"라는 단어 1개

그림 2-5 단어 "you"의 맥락에 포함되는 단어의 빈도를 표로 정리한다.

	you	say	goodbye	and	i	hello	.
you	0	1	0	0	0	0	0

- 표로 표현
- 따라서 'you'라는 단어를 [0,1,0,0,0,0,0]이라는 벡터로 표현할 수 있다.

그림 2-7 모든 단어 각각의 맥락에 해당하는 단어의 빈도를 세어 표로 정리한다.

	you	say	goodbye	and	i	hello	.
you	0	1	0	0	0	0	0
say	1	0	1	0	1	1	0
goodbye	0	1	0	1	0	0	0
and	0	0	1	0	1	0	0
i	0	1	0	1	0	0	0
hello	0	1	0	0	0	0	1
.	0	0	0	0	0	1	0

→ 다른 단어들에 대해서도 수행한 결과

- 위의 표가 행렬의 형태를 가짐 → 동시발생 행렬

```

C=np.array([
    [0,1,0,0,0,0,0],
    [1,0,1,0,1,1,0],
    [0,1,0,1,0,0,0],
    [0,0,1,0,1,0,0],
    [0,1,0,1,0,0,0],
    [0,1,0,0,0,0,1],
    [0,1,0,0,0,1,0],
],dtype=np.int32)
print(C[0])
print(C[4])
print(C[word_to_id['goodbye']])

[0 1 0 0 0 0 0]
[0 1 0 1 0 0 0]
[0 1 0 1 0 0 0]

```

→ 동시발생 행렬을 사용한다면 다음과 같은 방식으로 각 단어의 벡터를 얻을 수 있다!

위와 같은 과정을 자동화해보자!

```

def create_co_matrix(corpus, vocab_size, window_size=1):
    corpus_size = len(corpus)
    co_matrix = np.zeros((vocab_size, vocab_size), dtype=np.int32)

    for idx, word_id in enumerate(corpus):
        for i in range(1, window_size + 1):
            left_idx = idx - i # left window_size
            right_idx = idx + i # right window_size

            if left_idx >= 0:
                left_word_id = corpus[left_idx]
                co_matrix[word_id, left_word_id] += 1

            if right_idx < corpus_size:
                right_word_id = corpus[right_idx]
                co_matrix[word_id, right_word_id] += 1

    return co_matrix

```

- co_matrix를 0으로 채워진 2차원 배열로 초기화

- 말뭉치의 모든 단어 각각에 대하여 윈도우에 포함된 주변 단어를 카운트

2.3.5 벡터 간 유사도

벡터 사이의 유사도 → 벡터의 내적 or 유클리드 거리

단어 벡터의 유사도 : 코사인 유사도

*두 벡터가 가리키는 방향이 얼마나 비슷한가? 완전히 같으면 1, 완전히 다르면 -1

$$\text{similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{x_1 y_1 + \dots + x_n y_n}{\sqrt{x_1^2 + \dots + x_n^2} \sqrt{y_1^2 + \dots + y_n^2}}$$

- 분자 : 벡터의 내적
- 분모 : 각 벡터의 노름 * 노름은 벡터의 크기, 여기서 L2노름을 계산(벡터의 각 원소를 제곱하여 더한 후, 다시 제곱근을 구하여 계산)
- 핵심 : 벡터 정규화, 내적 계산

```
def cos_similarity(x, y, eps=1e-8):
    nx = x / np.sqrt(np.sum(x**2) + eps) # x의 정규화
    ny = y / np.sqrt(np.sum(y**2) + eps) # y의 정규화
    return np.dot(nx, ny)
```

- 위와 같은 함수는 먼저 벡터 x 와 y를 정규화한 후 두 벡터의 내적을 구함
 - 인수로 제로 벡터가 들어오면 0으로 나누는 오류가 발생
- 분모에 작은 값을 더해주어 해결

```

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)

c0 = C[word_to_id['you']] # "you"의 단어 벡터
c1 = C[word_to_id['i']] # 'i'의 단어 벡터
print(cos_similarity(c0, c1))

0.7071067758832467

```

- 'you'와 'I'의 코사인 유사도는 0.70이 나온다! (유사성이 크다)

2.3.6 유사 단어의 랭킹 표시

- 단어가 검색어로 주어지면, 검색어와 비슷한 단어를 유사도 순으로 출력하는 함수

```
most_similar(query, word_to_id, id_to_word, word_matrix, top=5)
```

표 2-1 most_similar() 함수의 인수

인수명	설명
query	검색어(단어)
word_to_id	단어에서 단어 ID로의 딕셔너리
id_to_word	단어 ID에서 단어로의 딕셔너리
word_matrix	단어 벡터들을 한데 모은 행렬. 각 행에는 대응하는 단어의 벡터가 저장되어 있다고 가정한다.
top	상위 몇 개까지 출력할지 설정

```

def most_similar(query, word_to_id, id_to_word, word_matrix, top=5):
    # 1) 검색어를 꺼낸다.
    if query not in word_to_id:
        print(f'{query}(을)를 찾을 수 없습니다.')
        return

    print(f'\n[query] {query}')
    query_id = word_to_id[query]
    query_vec = word_matrix[query_id]

    # 2) 코사인 유사도 계산
    vocab_size = len(id_to_word)
    similarity = np.zeros(vocab_size)
    for i in range(vocab_size):
        similarity[i] = cos_similarity(word_matrix[i], query_vec)

    # 3) 코사인 유사도를 기준으로 내림차순으로 출력
    count = 0
    for i in (-1 * similarity).argsort():
        if id_to_word[i] == query:
            continue
        print(f' {id_to_word[i]}: {similarity[i]}')

        count += 1
        if count >= top:
            return

```

위 코드 동작 순서

1. 검색어의 단어 벡터를 꺼낸다.
2. 검색어의 단어 벡터와 다른 모든 단어 벡터와의 코사인 유사도를 각각 구한다.
3. 계산한 코사인 유사도 결과를 기준으로 값이 높은 순서대로 출력한다.
 - similarity 배열에 담긴 원소의 인덱스를 내림차순으로 정렬한 후 상위 원소들을 출력
 - argsort() : 배열의 원소를 오름차순으로 정리
 - 이 메서드를 사용하여 단어의 유사도가 '큰' 순서대로 정렬할 수 있다.

```

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)

most_similar('you', word_to_id, id_to_word, C, top=5)

[query] you
goodbye: 0.7071067758832467
i: 0.7071067758832467
hello: 0.7071067758832467
say: 0.0
and: 0.0

```

- 'you' 와 유사한 단어 상위 5개만 출력
- 'i', 'you' 모두 인칭 대명사이기 때문에 유사도가 높은 것은 이해가 가지만, 'goodbye', 'hello'와의 유사도가 높은 것은 말뭉치의 크기가 너무 작기 때문에 발생한 것

2.4 통계 기반 기법 개선하기

2.4.1 상호정보량

- 동시발생 행렬의 원소 $\Rightarrow$ 두 단어가 동시에 발생한 횟수를 나타냄
- '발생' 횟수
 $\Rightarrow$ 발생횟수가 많다는 것은 단순히 고빈도 단어라 특정 단어와의 관련성이 높게 나올 가능성이 있다.

위의 문제를 해결하기 위한 방법 : 점별 상호정보량(PMI)

$$PMI(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)} \quad \text{[식 2.2]}$$

- $P(x)$: x 가 일어날 확률 (단어 x 가 말뭉치에서 등장할 확률)
- $P(y)$: y 가 일어날 확률

- $P(x,y)$: x 와 y 가 동시에 일어날 확률 (단어 x,y 가 동시발생할 확률)
- PMI 값이 높다 → 관련성이 높다

$$PMI(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)} = \log_2 \frac{\frac{C(x,y)}{N}}{\frac{C(x)}{N} \frac{C(y)}{N}} = \log_2 \frac{C(x,y) \cdot N}{C(x)C(y)} \quad [\text{식 2.3}]$$

- 동시발생행렬 C 를 사용하여 다시 식 계산

ex) 단어 수 (N)=10000, 'the' 와 'car'와 'drive'가 각각 1000번 20번 10번 등장한다고 가정하면, 동시발생 횟수 관점에서는 'car'는 'drive'보다는 'the'와의 관련이 깊다고 나온다.

PMI 계산 →

$$PMI(\text{"the"}, \text{"car"}) = \log_2 \frac{10 \cdot 10000}{1000 \cdot 20} \approx 2.32 \quad [\text{식 2.4}]$$

$$PMI(\text{"car"}, \text{"drive"}) = \log_2 \frac{5 \cdot 10000}{20 \cdot 10} \approx 7.97 \quad [\text{식 2.5}]$$

PMI를 사용하면 'car'는 'the'보다 'drive'와의 관련성이 더 강해진다.

⇒ 단어가 단독으로 출현하는 횟수가 고려되었기 때문!

⇒ 'the'가 더 자주 출현하였기 때문에 PMI 점수가 낮아졌다.

PMI의 문제점

- 두 단어의 동시발생 횟수가 0이면 $\log_2 0$ 이 되어서 -무한대로 발산하게됨.

⇒ 양의 상호정보량 **PPMI** 를 사용!

$$PPMI(x,y) = \max(0, PMI(x,y)) \quad [\text{식 2.6}]$$

- PMI가 음수일때는 0으로 취급!

```

def ppmi(C, verbose=False, eps=1e-8):
    '''PPMI(점별 상호정보량) 생성
    :param C: 동시발생 행렬
    :param verbose: 진행 상황을 출력할지 여부
    :return: ppmi
    '''
    M = np.zeros_like(C, dtype=np.float32)
    N = np.sum(C) # num of corpus
    S = np.sum(C, axis=0) # 각 단어의 출현 횟수
    total = C.shape[0] * C.shape[1]
    cnt = 0

    for i in range(C.shape[0]):
        for j in range(C.shape[1]):
            pmi = np.log2(C[i, j] * N / (S[i]*S[j]) + eps)
            M[i, j] = max(0, pmi)

            if verbose:
                cnt += 1
                if cnt % (total//100) == 0:
                    print(f' {(100*cnt/total):.2f} 완료')

    return M

```

- C : 동시발생 행렬
- verbose : 진행상황 출력 여부

⇒ 단어 x와 y가 동시에 발생하는 횟수를 C(x,y)라고 했을 때,

$$C(x) = \sum_i C(i,x), \quad C(y) = \sum_i C(i,y), \quad N = \sum_i \sum_j C(i,j)$$

이 되도록 구현 (근삿값을 구하도록)

```

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)
W=ppmi(C)

np.set_printoptions(precision=3)
print('동시발생 행렬')
print(C)
print('-'*50)
print('PPMI')
print(W)

```

동시발생 행렬

```

[[0 1 0 0 0 0 0]
 [1 0 1 0 1 1 0]
 [0 1 0 1 0 0 0]
 [0 0 1 0 1 0 0]
 [0 1 0 1 0 0 0]
 [0 1 0 0 0 0 1]
 [0 0 0 0 0 1 0]]

```

PPMI

```

[[0.      1.807 0.      0.      0.      0.      0.      ]
 [1.807 0.      0.807 0.      0.807 0.807 0.      ]
 [0.      0.807 0.      1.807 0.      0.      0.      ]
 [0.      0.      1.807 0.      1.807 0.      0.      ]
 [0.      0.807 0.      1.807 0.      0.      0.      ]
 [0.      0.807 0.      0.      0.      0.      2.807]
 [0.      0.      0.      0.      0.      2.807 0.      ]]

```

- 동시발생 행렬 → PPMI 행렬로 변환

하지만, PPMI 행렬에도 문제가 존재!

→ 말뭉치의 어휘 수가 증가함에 따라 각 단어 벡터의 차원 수도 증가

→ 원소 대부분이 0인것을 확인할 수 있다 → 각 원소의 '중요도'가 낮다

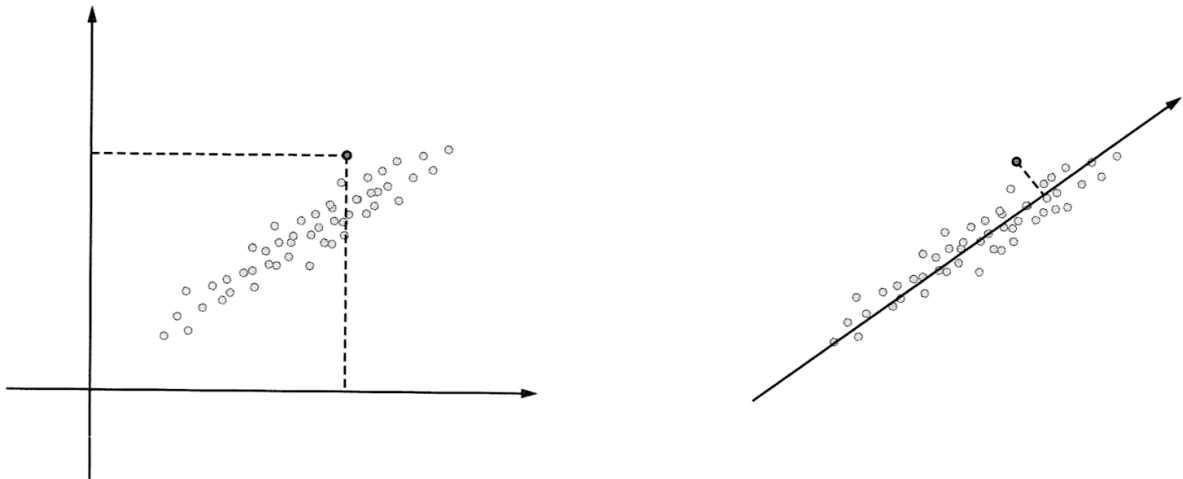
해결하기 위한 방법 ⇒ *차원 감소

2.4.2 차원 감소

차원 감소

- 벡터의 차원을 줄이는 방법
 - '중요한 정보'는 최대한 유지하면서 줄이는 게 핵심

그림 2-8 그림으로 이해하는 차원 감소: 2차원 데이터를 1차원으로 표현하기 위해 중요한 축(데이터를 넓게 분포시키는 축)을 찾는다.



- 이렇게 데이터를 표현할 수 있는 가장 적합한 축을 찾아낸다!
→ 1차원 값만으로도 데이터의 본질적인 차이를 구별할 수 있어야한다!

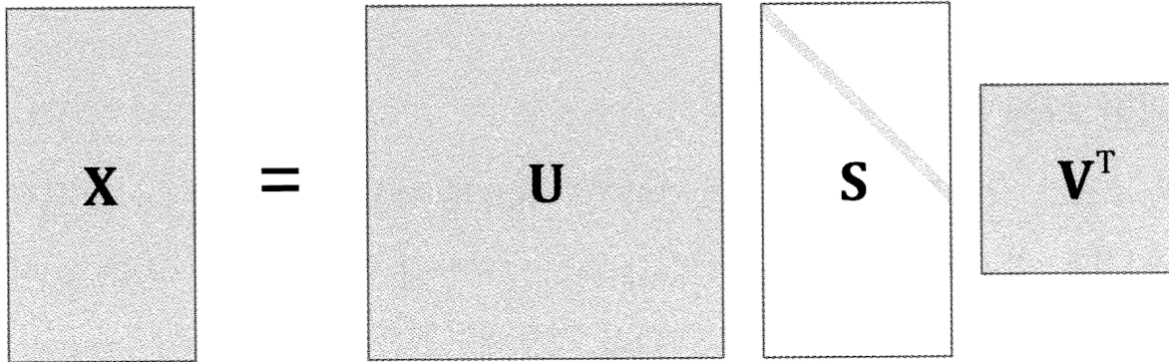
차원을 감소시키는 방법 : **특이값 분해 (SVD)**

$$\mathbf{X} = \mathbf{USV}^T$$

[식 2.7]

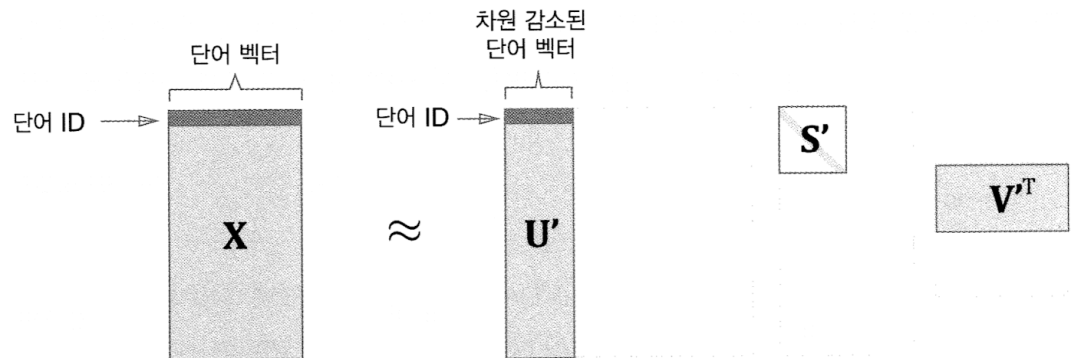
- 임의의 행렬을 3개의 행렬의 곱으로 분해
- U, V는 직교행렬, S는 대각행렬

그림 2-9 SVD에 의한 행렬의 변환(행렬의 '흰 부분'은 원소가 0임을 뜻함)



- U 행렬 : '단어 공간'
- S : 대각행렬, 대각성분에는 '특이값'이 큰 순서대로 나열되어있다.
 - 특이값이란? → '해당 축'의 중요도

그림 2-10 SVD에 의한 차원 감소



- 위의 그림과 같이 중요도가 낮은 원소(특이값이 작은 원소)를 깎아내는 방법으로 차원 축소
- 행렬 U에서 여분의 열벡터를 깎아내어 원래의 행렬을 근사할 수 있다
 - '단어의 PPMI 행렬'에 적용
 - 행렬 X의 각 행에는 해당 단어 ID의 단어 벡터가 저장되어 있고, 그 단어 벡터가 행렬 U' 라는 차원 감소된 벡터로 표현된다.

2.4.3 SVD에 의한 차원 감소

동시발생 행렬을 만들어 PPMI 행렬로 변환한 다음 SVD 적용

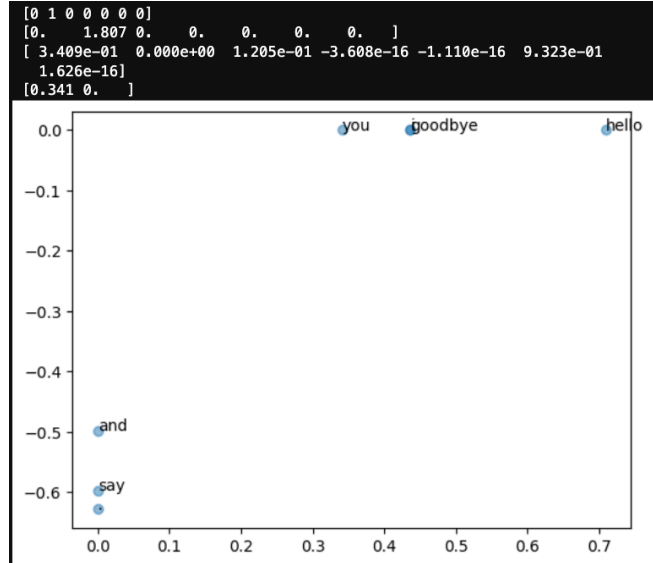
```
import numpy as np
import matplotlib.pyplot as plt

text = 'You say goodbye and I say hello.'
corpus, word_to_id, id_to_word = preprocess(text)
vocab_size = len(word_to_id)
C = create_co_matrix(corpus, vocab_size)
W = ppmi(C)

# SVD
U, S, V = np.linalg.svd(W)

print(C[0]) # 동시발생 행렬
print(W[0]) # PMI 행렬
print(U[0]) # SVD
# 2차원으로 차원 축소하기
print(U[0, :2])

# 글꼴
for word, word_id in word_to_id.items():
    plt.annotate(word, (U[word_id, 0], U[word_id, 1]))
plt.scatter(U[:, 0], U[:, 1], alpha=0.5)
plt.show()
```



- SVD에 의해 변환된 밀집벡터 표현은 변수 U에 저장
 - 단어 ID가 0인 단어 벡터를 살펴보면
 - 원래는 희소벡터인 W[0]이 SVD에 의해서 밀집벡터 U[0]으로 변했다.
- 다음으로 2차원 그래프상에서 좌표 지점에 word에 담긴 텍스트를 그림 (i, goodbye 겹쳐있음)
 - 'goodbye'와 'hello', 'you'와 'i'가 가까이 있는 것을 확인할 수 있다.

⇒ PTB라는 데이터셋을 활용하여 더 큰 말뭉치를 사용하여 똑같은 실험 진행

2.4.4 PTB 데이터셋

펜 트리뱅크 (PTB) : 본격적인 말뭉치 (너무 크지도 작지도 않은 말뭉치)

그림 2-12 PTB 말뭉치(텍스트 파일)의 예

```
1 consumers may want to move their telephones a little closer to the tv set
2 <unk> <unk> watching abc 's monday night football can now vote during <unk> for the greatest play in N years from
- among four or five <unk> <unk>
3 two weeks ago viewers of several nbc <unk> consumer segments started calling a N number for advice on various
- <unk> issues
4 and the new syndicated reality show hard copy records viewers ' opinions for possible airing on the next day 's show
5 interactive telephone technology has taken a new leap in <unk> and television programmers are racing to exploit the
- possibilities
6 eventually viewers may grow <unk> with the technology and <unk> the cost
```

- PTB 말뭉치에서는 한 문장이 하나의 줄로 저장되어있다.

⇒ 각 문장을 연결한 '하나의 큰 시계열 데이터'로 취급

```

import sys
sys.path.append('.')
from dataset import ptb

corpus, word_to_id, id_to_word = ptb.load_data('train')

print('말뭉치 크기:', len(corpus))
print('corpus[:30]:', corpus[:30])
print()
print('id_to_word[0]:', id_to_word[0])
print('id_to_word[1]:', id_to_word[1])
print('id_to_word[2]:', id_to_word[2])
print()
print("word_to_id['car']:", word_to_id['car'])
print("word_to_id['happy']:", word_to_id['happy'])
print("word_to_id['lexus']:", word_to_id['lexus'])

```

```

말뭉치 크기: 929589
corpus[:30]: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29]

id_to_word[0]: aer
id_to_word[1]: banknote
id_to_word[2]: berlitz

word_to_id['car']: 3856
word_to_id['happy']: 4428
word_to_id['lexus']: 7426

```

말뭉치를 다루는 방법

- corpus에는 단어 ID 목록이 저장
- id_to_word는 단어 ID에서 단어로 변환하는 딕셔너리, word_to_id는 단어에서 단어 ID로 변환하는 딕셔너리

2.4.5 PTB 데이터셋 평가

PTB 데이터셋에 통계기반 기법 적용 → 고속 SVD 사용

```

import numpy as np
from dataset import ptb

window_size = 2
wordvec_size = 100

corpus, word_to_id, id_to_word = ptb.load_data('train')
vocab_size = len(word_to_id)
print('Create Co-occurrence Matrix...')
C = create_co_matrix(corpus, vocab_size, window_size)

print('PPMI 계산...')
W = ppmi(C, verbose=True)

try:
    # truncated SVD
    from sklearn.utils.extmath import randomized_svd
    U, S, V = randomized_svd(W, n_components=wordvec_size, n_iter=5,
                             random_state=None)
except:
    # SVD
    U, S, V = np.linalg.svd(W)

word_vecs = U[:, :wordvec_size]
querys = ['you', 'year', 'car', 'toyota']
for query in querys:
    most_similar(query, word_to_id, id_to_word, word_vecs, top=5)

```

```
[query] you
i: 0.6429193019866943
we: 0.5806225538253784
anybody: 0.5760573148727417
someone: 0.5607318878173828
do: 0.5547293424606323

[query] year
month: 0.7172791957855225
earlier: 0.6525256633758545
last: 0.6344152688980103
next: 0.6183114647865295
quarter: 0.603631317615509

[query] car
auto: 0.7378795146942139
luxury: 0.7202755212783813
cars: 0.5880320072174072
truck: 0.5599579811096191
corsica: 0.5532428026199341

[query] toyota
motor: 0.7376506328582764
motors: 0.6657252311706543
lexus: 0.646793007850647
honda: 0.6128685474395752
nissan: 0.603904128074646
```

- 각 단어에 대해서 유사도가 높은 단어들이 상위를 차지했음을 알 수 있다.
 - you - i, we, anybody ..
 - year - month, earlier, last ..
 - car - auto, luxury ..
 - toyota - motor, motors ..

⇒ 사람의 직관과 비슷한 결과!

2.5 정리



- WordNet 등의 시소러스를 이용하면 유의어를 얻거나 단어 사이의 유사도를 측정하는 등 유용한 작업을 할 수 있다.
- 시소러스 기반 기법은 시소러스를 작성하는 데 엄청난 인적 자원이 들고, 변화해가는 새로운 단어에 대응하기 어렵다.
- 현재는 말뭉치를 이용해 단어를 벡터화하는 방식이 주로 쓰인다.
- 최근의 단어 벡터화 기법들은 대부분 '단어의 의미는 주변 단어에 의해 형성된다'는 분포 가설에 기초한다.
- 통계 기반 기법은 말뭉치 안의 각 단어에 대해서 그 단어의 주변 단어의 빈도를 집계한다.
- 동시발생 행렬을 PPMI 행렬로 변환하고 다시 차원을 감소시킴으로써, 거대한 '희소벡터'를 작은 '밀집벡터'로 변환할 수 있다.
- 단어의 벡터 공간에서는 의미가 가까운 단어는 그 거리도 가까울 것으로 기대된다.